

A IMPORTÂNCIA DA VISÃO ARQUITETURAL NA PROGRAMAÇÃO

LOW-CODE: UM ESTUDO SOBRE QUALIDADE SISTÊMICA

THE IMPORTANCE OF ARCHITECTURAL VISION IN LOW-CODE PROGRAMMING: A STUDY ON SYSTEMIC QUALITY

Juraci Nunes Tito Neto - UNOPAR

titojneto@gmail.com

Resumo. Este artigo analisa a relevância dos padrões arquiteturais no desenvolvimento *Low-Code*. Investiga-se como ferramentas de alta produtividade demandam aplicação rigorosa de conceitos como Modularidade e Separação de Preocupações (SoC). Sob a ótica da norma ISO/IEC 25010, discute-se o impacto do design na manutenibilidade e segurança sistêmica. O objetivo é demonstrar que a robustez arquitetural é indispensável para mitigar dívidas técnicas e *dark patterns*. Conclui-se que a visão estrutural é o alicerce necessário para converter a agilidade do alto nível em software empresarial resiliente.

Palavras-chave: arquitetura de software; padrões de design; low-code; qualidade de software; engenharia de software.

Abstract. This article analyzes the relevance of architectural patterns in Low-Code development. It investigates how high-productivity tools demand the rigorous application of concepts such as Modularity and Separation of Concerns (SoC). Under the perspective of the ISO/IEC 25010 standard, the impact of design on systemic maintainability and security is discussed. The objective is to demonstrate that architectural robustness is indispensable for mitigating technical debts and dark patterns. It concludes that structural vision is the necessary foundation for converting high-level agility into resilient enterprise software.

Keywords: software architecture; design patterns; low-code; software quality; software engineering.

1 Introdução

O cenário contemporâneo do desenvolvimento de software é marcado pela ascensão das plataformas de alta abstração, denominadas *Low-Code*, que evoluíram do aperfeiçoamento dos serviços de PaaS (*Platform as a Service*) para atender à demanda por agilidade em empresas de diversos portes. Essas ferramentas permitem a entrega acelerada de interfaces e fluxos de trabalho por meio da redução do esforço de codificação manual. Todavia, essa agilidade superficial não substitui a necessidade indispensável de um design de software sólido e bem estruturado.

A problemática central reside no fato de que, sem uma visão arquitetural de alto nível, os sistemas construídos sob este paradigma tendem a se tornar frágeis, de difícil manutenção e incapazes de escalar. A facilidade de "arrastar e soltar" componentes pode ocultar complexidades críticas e déficits de conhecimentos básicos de engenharia, resultando em dívidas técnicas e vulnerabilidades que comprometem a longevidade da solução. Além disso, a ascensão do *Citizen Developer* — usuário sem formação técnica — amplia o risco de TI Invisível (*Shadow IT*), gerando sistemas isolados e redundantes que podem comprometer a integridade dos dados corporativos.

Neste contexto, o presente artigo investiga a relevância crítica dos padrões arquiteturais e do design de software na era *Low-Code*. Através da lente dos Atributos de Qualidade da norma ISO/IEC 25010, o estudo discute o impacto de decisões estruturais na manutenibilidade e segurança sistêmica. O objetivo central é demonstrar que a construção de ecossistemas digitais

de alta performance depende da manutenção de uma arquitetura robusta, documentada e resiliente, garantindo que a facilidade de desenvolvimento não comprometa a robustez sistêmica.

2 Metodologia

O presente estudo caracteriza-se como uma pesquisa de natureza estritamente teórica e exploratória, conduzida por meio de uma revisão bibliográfica e análise comparativa qualitativa. O procedimento metodológico consistiu no levantamento e na sistematização de conceitos fundamentais da Engenharia de Software e da Governança de TI, visando analisar sua aplicabilidade no paradigma de desenvolvimento *Low-Code*.

A fundamentação teórica baseou-se em três pilares principais, cujas definições norteiam a análise qualitativa deste estudo:

Etapa 1: Levantamento e Enquadramento Normativo-Arquitetural: Esta etapa consistiu na sistematização de conceitos de Modelagem Arquitetural baseada no Modelo de Visões 4+1 de Kruchten. O objetivo foi organizar o design de software em cinco perspectivas complementares — Lógica, Processo, Desenvolvimento, Física e Cenários — para gerenciar a complexidade sistêmica de forma holística, independentemente da ferramenta de implementação utilizada. Paralelamente, adotou-se a norma internacional ISO/IEC 25010 como métrica para avaliar os atributos de Adequação Funcional, Eficiência de Desempenho, Compatibilidade, Usabilidade, Confiabilidade, Segurança, Manutenibilidade e Portabilidade. Esta norma estabelece o guia fundamental para a engenharia de requisitos e para a validação sistêmica, definindo a adequação e a eficiência do produto final através de suas oito características principais. A união dessas abordagens permitiu contrastar o desenvolvimento *Low-Code* sob a ótica do rigor técnico e da governança estratégica.

Etapa 2: Definição de Métricas e Normatização de Qualidade: Esta etapa consistiu na adoção da norma internacional ISO/IEC 25010 como métrica fundamental para avaliar os atributos de Adequação Funcional, Eficiência de Desempenho, Compatibilidade, Usabilidade, Confiabilidade, Segurança, Manutenibilidade e Portabilidade. A norma estabelece um guia para a engenharia de requisitos e para a validação sistêmica, sendo composta por oito características principais que definem a adequação e a eficiência do produto final. No contexto deste estudo, o foco recaiu sobre a avaliação de atributos críticos para o paradigma *Low-Code*, especificamente Manutenibilidade, Segurança, Capacidade de Expansão (Escalabilidade) e Portabilidade, visando identificar como o design estratégico mitiga vulnerabilidades e dívidas técnicas estruturais.

Etapa 3: Análise de Complexidade e Governança Estratégica: Esta etapa consistiu na aplicação da Complexidade Ciclômática de McCabe (1976) que estabelece que uma complexidade ciclômática superior a 10 indica um módulo de alto risco e difícil manutenção; no contexto *Low-Code*, tal valor caracteriza o “Monólito Visual”, onde a lógica emaranhada impede a evolução do sistema. Destaque para o seu uso prático, onde a métrica foi usada para mensurar o número de caminhos logicamente independentes nos fluxos visuais, permitindo identificar processos não modulares que elevam o risco de falhas e dificultam a testabilidade exaustiva. A análise foi integrada aos princípios de Governança de TI de Weill e Ross, que estabelecem a necessidade de decisões arquiteturais explícitas e padronizadas para evitar que a agilidade das novas tecnologias seja consumida pela complexidade e pelos custos de integração de sistemas

A IMPORTÂNCIA DA VISÃO ARQUITETURAL NA PROGRAMAÇÃO LOW-CODE: UM ESTUDO SOBRE QUALIDADE SISTÊMICA

incompatíveis. Portanto, essa abordagem permitiu validar o papel do arquiteto como um Gestor da Qualidade Sistêmica, capaz de utilizar padrões como *Adapter*, *Strategy* e *Strangler Fig* para mitigar a dívida técnica estrutural e garantir a resiliência do ecossistema digital global independentemente da tecnologia de desenvolvimento operada (plataforma de alta abstração low-code e similares, ou modelo de IA geradoras de código).

A análise dos dados foi estruturada através do contraste técnico entre três cenários de desenvolvimento: o tradicional (codificação manual), o *Low-Code* sem governança e o *Low-Code* orientado por design arquitetural. Para validar a robustez das soluções propostas, foram examinados padrões de design clássicos, como *Adapter*, *Strategy* e *Strangler Fig*, sob a ótica da redução da dívida técnica e mitigação da complexidade ciclomática de McCabe.

3 Resultados e Discussão

Os resultados desta pesquisa demonstram que a eficácia de sistemas em ambientes de alta abstração é diretamente proporcional à robustez da arquitetura subjacente. A análise comparativa revelou disparidades críticas entre o desenvolvimento puramente visual e aquele orientado por design estratégico, evidenciando que a ausência de uma visão estrutural transforma a agilidade inicial em uma fragilidade de longo prazo.

3.1 Análise Comparativa de Abordagens

Sob a perspectiva da governança corporativa, sustenta-se que a autonomia concedida pelas plataformas de alta produtividade não deve ser dissociada de uma arquitetura padronizada, sob o risco de criar silos tecnológicos que elevam os custos de integração e manutenção. Nesse sentido, a literatura reforça que o valor estratégico de novas tecnologias é rapidamente erodido se as decisões de design forem tomadas de forma isolada e sem o devido rigor técnico.

Ademais, as análises sugerem que a qualidade sistêmica em projetos de alta abstração reside na capacidade do arquiteto de projetar além da superfície, utilizando modelos que permitam gerenciar a complexidade através de múltiplas dimensões e abstrações complementares. Portanto, infere-se que o diferencial competitivo não se encontra na ferramenta de implementação em si, mas na inteligência do design de software, que atua como o alicerce indispensável para converter a velocidade do *Low-Code* em sistemas resilientes, seguros e de nível empresarial.

A Tabela 1 sintetiza o impacto da visão arquitetural nos principais atributos de entrega e manutenção. Observa-se que, embora o *Low-Code* sem governança ofereça a maior velocidade inicial, ele compromete a escalabilidade e a resiliência ao risco a longo prazo.

Tabela 1: Comparativo de Atributos de Qualidade por Abordagem

Atributo de Qualidade	Desenvolvimento Tradicional	Low-Code (Sem Arquiteto)	Low-Code Arquitetado
Velocidade	Moderada (Codificação manual)	Altíssima (Foco na interface)	Alta (Agilidade com governança)
Manutenibilidade	Depende do rigor do código	Baixa (Caos de componentes)	Alta (Modularização estratégica)
Escalabilidade	Planejada no Core	Limitada à ferramenta	Garantida via APIs externas
Resiliência	Dívida técnica no código	Dívida técnica estrutural	Mitigado por Padrões e ADRs

Fonte: Elaborada pelo autor (2026).

Conforme evidenciado na análise comparativa, o cenário de desenvolvimento *ad-hoc* resulta em um acúmulo acelerado de dívidas técnicas. A baixa manutenibilidade observada no desenvolvimento *ad-hoc* decorre diretamente da negligência da Visão de Desenvolvimento do Modelo 4+1 de Kruchten, onde a integridade estrutural é sacrificada em prol da agilidade visual imediata. Sem o mapeamento claro de subsistemas e a gestão de dependências nesta visão, o sistema perde sua capacidade de evolução.

O uso de *Architectural Decision Records* (ADRs) revela-se vital neste cenário, pois documenta as justificativas das escolhas técnicas em um formato estruturado e histórico. Em ecossistemas *Low-Code*, onde a facilidade de alteração visual pode induzir a modificações impulsivas, as ADRs garantem que o conhecimento estratégico não se perca na volatilidade das interfaces. Essa prática atua como um mecanismo de defesa contra a erosão arquitetural, assegurando que decisões sobre integrações ou padrões de segurança sejam preservadas, facilitando a manutenibilidade e a continuidade do ciclo de vida do sistema, conforme preconizado pelos atributos de resiliência analisados

Como destacado na análise, a ausência de uma estrutura modular no cenário sem arquiteto compromete diretamente a evolução do sistema, gerando o que se denomina "Monólito Visual". Sobre este risco, a literatura técnica oferece uma métrica de controle fundamental:

A utilização da Complexidade Ciclomática de McCabe (1976) valida a necessidade de rigor técnico ao revelar que fluxos visuais não modulares resultam em um crescimento exponencial de caminhos lógicos independentes. Segundo McCabe, essa medida de complexidade é fundamental para determinar a testabilidade e a confiabilidade do software; em plataformas *Low-Code*, se o design não for modular, o sistema torna-se impossível de ser testado de forma exaustiva, elevando drasticamente o risco de falhas catastróficas em ambiente de produção.

Essa fundamentação explica por que a governança não é apenas uma escolha estética, mas uma necessidade de sobrevivência do software corporativo. A partir dessa perspectiva, nota-se que o papel do arquiteto sofre uma mudança de escopo de atuação, em que ele deixa de ser um mero implementador para se tornar um Gestor da Qualidade Sistêmica.

A IMPORTÂNCIA DA VISÃO ARQUITETURAL NA PROGRAMAÇÃO LOW-CODE: UM ESTUDO SOBRE QUALIDADE SISTÊMICA

3.2 Análise dos Atributos ISO/IEC 25010

A aplicação da norma ISO/IEC 25010 permitiu identificar que as características de Manutenibilidade e Portabilidade são as mais vulneráveis no paradigma de alta abstração. A Tabela 2 demonstra como o design profissional "externaliza" a complexidade crítica para proteger o sistema.

Tabela 2: Impacto da Visão Arquitetural sob a Ótica da ISO 25010

Característica (ISO 25010)	Desenvolvimento Low-Code Sem Design	Low-Code com Visão Arquitetural (Estratégico)	Impacto na Qualidade Sistemática
Manutenibilidade	Baixa: Alterações lógicas exigem modificações diretas na interface, gerando acoplamento.	Alta: Uso de camadas de serviço isoladas, permitindo evolução sem impacto na UI.	Facilita a correção de falhas e a implementação de melhorias.
Segurança	Dependente: Limitada às configurações padrão do fornecedor da plataforma.	Reforçada: Implementação de camadas externas de autenticação e criptografia (ex: JWT/OAuth).	Protege a integridade dos dados e a confiança do usuário.
Escalabilidade	Limitada: O sistema degrada conforme o volume de dados aumenta dentro da ferramenta.	Garantida: Integração via microsserviços e processamento assíncrono para tarefas pesadas.	Suporta o crescimento do negócio sem necessidade de reescrita.
Portabilidade	Nula: O sistema sofre de <i>vendor lock-in</i> , ficando preso à plataforma original.	Mitigada: Uso de Padrões Adaptadores que facilitam a migração de lógica para outros ambientes.	Reduz o risco estratégico de dependência tecnológica única.

Fonte: Elaborada pelo autor (2026).

A aplicação da visão arquitetural sobre as características da norma ISO/IEC 25010 revela que o diferencial de um sistema de alta performance não reside na ferramenta, mas na capacidade do arquiteto de isolar a complexidade. Abaixo, detalham-se os resultados observados:

Manutenibilidade: O uso de camadas de serviço isoladas no *Low-Code* estratégico permite a evolução do sistema sem impacto na interface de usuário (UI). Esta prática corrobora o princípio da Separação de Preocupações (SoC), garantindo que a lógica de negócio permaneça independente e modular. De acordo com a perspectiva de Kruchten (1995), essa organização modular é o que permite gerenciar a evolução do sistema a longo prazo, reduzindo o custo total de propriedade.

Segurança: A implementação de camadas externas de autenticação (como JWT ou OAuth) reforça a proteção para além das configurações padrão fornecidas pela plataforma. Essa decisão arquitetural é fundamental para garantir a integridade e a autenticidade dos dados em ecossistemas distribuídos. Conforme defendido por Weill e Ross (2006), a segurança não deve ser uma concessão da ferramenta, mas uma política de governança integrada a uma arquitetura corporativa padronizada que protege o patrimônio digital da organização.

Portabilidade: O emprego de padrões *Adapter* atua como uma salvaguarda contra o *vendor lock-in*, permitindo que a organização mantenha a soberania sobre seus processos caso necessite mitiga o risco latente, facilitando a migração de lógica para outros ambientes ou serviços. Esta camada de abstração protege o sistema contra a obsolescência forçada e garante a interoperabilidade entre plataformas distintas. A literatura de governança reforça que a padronização das interfaces e o desacoplamento estratégico são os elementos que permitem a liberdade de inovação e a flexibilidade tecnológica necessária em cenários de mudança rápida.

Porquanto, a aplicação da visão arquitetural sobre as características da norma ISO/IEC 25010 garante a interoperabilidade entre plataformas distintas sem abrir brechas para déficits técnicos que comprometam a criação, manutenção e eventuais correções da aplicação. A Tabela 3 sintetiza como o perfil do desenvolvedor e a governança aplicada influenciam diretamente a integridade e o controle do ecossistema digital. A análise demonstra que a ausência de uma supervisão arquitetural transforma a agilidade do *Low-Code* em um multiplicador de riscos operacionais. Enquanto o desenvolvimento leigo foca na entrega imediata de funcionalidades, a visão estratégica do arquiteto assegura que cada componente visual adira aos padrões de segurança e manutenibilidade, convertendo ferramentas de alta abstração em ativos de software sustentáveis para a organização.

Tabela 3: Síntese de Impactos por Ator de Desenvolvimento e Governança

Ator do Desenvolvimento	Abordagem Técnica	Impacto no Ecossistema	Métrica de Controle
Citizen Developer	Ad hoc / Visual puro	Risco de Silos e Shadow IT	Pontos de Função
Arquiteto de Software	Low-Code Estratégico	Sistemas resilientes e escaláveis	Complexidade Ciclomática

. Fonte: Elaborada pelo autor (2026).

3.3 Discussão Técnica e Validação de Modelos

A discussão dos resultados aponta que a "agilidade superficial" das plataformas *Low-Code* induz a uma falsa sensação de produtividade ou sucesso imediato, ocultando dívidas técnicas estruturais graves que emergem criticamente apenas na fase de operação e evolução do sistema. Sob a ótica da governança tecnológica, a eficiência de uma solução não deve ser mensurada apenas pela velocidade de entrega, mas pela resiliência da arquitetura em suportar integrações e mudanças sem a degradação da performance original.

Nesse sentido, a aplicação do Modelo de Visões 4+1 revela-se vital para a separação de interesses, onde a Visão Lógica deve permanecer rigorosamente independente da camada de apresentação para garantir que a integridade da lógica de negócio não seja comprometida por restrições da interface. A literatura reforça que o design de sistemas complexos não pode ser capturado em uma única dimensão, exigindo abstrações complementares que permitam ao arquiteto gerenciar requisitos conflitantes de forma holística.

A IMPORTÂNCIA DA VISÃO ARQUITETURAL NA PROGRAMAÇÃO LOW-CODE: UM ESTUDO SOBRE QUALIDADE SISTÊMICA

Sob essa ótica, a Visão de Desenvolvimento atua como o pilar de organização modular, essencial para evitar o acoplamento excessivo que frequentemente degrada projetos de alta abstração. Complementarmente, a Visão de Processo e a Visão Física asseguram que requisitos não-funcionais, como escalabilidade e performance em nuvem, sejam tratados como decisões de design deliberadas e não como subprodutos acidentais da ferramenta utilizada. Conforme estabelecido por Kruchten (1995), é a integração dessas múltiplas perspectivas, validadas por cenários de uso reais, que permite a construção de um plano de evolução resiliente para o sistema, garantindo sua longevidade e a redução do custo total de propriedade.

A utilização da Complexidade Ciclômática de McCabe valida essa necessidade técnica ao revelar que fluxos visuais não modulares resultam em um crescimento exponencial de caminhos lógicos independentes, tornando o software impossível de ser testado exaustivamente e elevando o risco de falhas catastróficas. Para quantificar a complexidade em ambientes de alta abstração, recomenda-se a adaptação de métricas tradicionais como:

Pontos de Função (APF): Essencial para medir a funcionalidade entregue ao negócio independentemente da tecnologia de "arrastar e soltar", permitindo estimativas de custo e produtividade reais.

Complexidade Ciclômática Adaptada: Utilizada para analisar o fluxo lógico visual; diagramas com excesso de ramificações indicam baixa testabilidade e alta propensão a erros operacionais.

Métricas de Acoplamento de APIs: Avaliam o volume de integrações externas; um alto índice de dependência sem o uso de Padrões Adapter sinaliza um risco elevado de *vendor lock-in* e fragilidade sistêmica.

Por fim, em última análise, a facilidade proporcionada pelas ferramentas de desenvolvimento de alto nível sugere que os *Citizen Developers* representam a última fronteira do desenvolvimento manual antes da automação total. No entanto, a partir de um breve diagnóstico amparado pela conclusão desta pesquisa, pode-se inferir que, embora o uso de ferramentas de alta abstração (low-code e no-code), e mais recentemente os Grandes Modelos de Linguagem (Large Language Models-LLMs), genericamente chamados de Inteligência Artificial (IA), facilitem a geração acelerada de fluxos, a ausência de uma visão arquitetural faz com que a velocidade de criação supere a capacidade de governança, automatizando dívidas técnicas em larga escala. Este novo paradigma introduz a "alucinação arquitetural", onde algoritmos generativos priorizam a funcionalidade imediata em detrimento da integridade estrutural, resultando na proliferação de fluxos logicamente opacos que carecem de padrões de segurança e modularidade. Sob a ótica da Governança de TI, a IA atua como um catalisador de complexidade: ao democratizar a automação, ela escala também as inconsistências de dados e os riscos de conformidade.

Por conseguinte, nesse cenário de hiper-automação, a supervisão técnica torna-se o único filtro capaz de traduzir o volume de entregas geradas sobretudo por IA, em ativos sistêmicos resilientes, evitando que a agilidade tecnológica se transforme em um legado de manutenção insustentável. Diante disso, o papel do arquiteto de software em contextos de abstração parcial (low-code e now-code) e/ou total (automação via IA geradoras de códigos) sofre uma metamorfose: ele deixa de ser um mero implementador para se tornar um Gestor da Qualidade Sistêmica concomitantemente em todos os níveis do desenvolvimento. De acordo com os preceitos de governança corporativa, é essa padronização arquitetural estratégica que, paradoxalmente, permite a liberdade de inovação nas pontas sem comprometer a robustez e a segurança do ecossistema digital global.

4 Conclusão

O presente estudo atingiu o objetivo proposto ao demonstrar que a ascensão das plataformas *Low-Code* e da filosofia de desenvolvimento *Citizen Developer* não torna obsoletos os fundamentos da Engenharia de Software; ao contrário, ela intensifica a necessidade de uma visão arquitetural rigorosa para garantir a sustentabilidade sistêmica. A pesquisa evidenciou que, embora a agilidade na entrega de interfaces seja um benefício imediato dessas tecnologias de alta abstração, a qualidade e a viabilidade de longo prazo dependem estritamente da aplicação de padrões consolidados e de uma governança técnica ativa.

Nesse sentido, a literatura de governança tecnológica reforça que a eficiência de uma solução digital não deve ser mensurada apenas pela celeridade de sua entrega, mas pela resiliência da arquitetura em suportar evoluções e integrações sem a degradação da performance original. De acordo com as perspectivas de Weill e Ross (2006), a governança de TI eficaz demanda que as decisões de design sejam integradas a uma arquitetura corporativa padronizada, evitando que a facilidade de implementação comprometa a integridade global do ecossistema.

Ademais, a análise fundamentada no Modelo de Visões 4+1 corrobora que o design de sistemas complexos exige o gerenciamento de múltiplas dimensões para garantir que a lógica de negócio permaneça preservada e independente das camadas de apresentação. Conforme estabelecido por Kruchten (1995), a visão arquitetural atua como o alicerce indispensável para gerenciar a complexidade através de abstrações que, somadas, determinam a longevidade e o custo total de propriedade do software. Conclui-se, portanto, que a união entre a velocidade da era *Low-Code* e do uso da *IA* e o rigor do planejamento estrutural é o único caminho para assegurar que a agilidade tecnológica se transforme em valor funcional de nível comercial.

Por fim, embora este estudo valide a eficácia dos padrões arquiteturais, as limitações residem na dependência da infraestrutura de terceiros típica do desenvolvimento em nuvem. Sugere-se para trabalhos futuros a investigação de frameworks de testes automatizados específicos para fluxos visuais, visando reduzir ainda mais a incidência de *dark patterns* operacionais em sistemas críticos.

5 Referências Bibliográficas

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR ISO/IEC 25010**: Engenharia de sistemas e software — Requisitos de qualidade e avaliação de sistemas e software (SQuaRE) — Modelos de qualidade de sistema e de produto de software. Rio de Janeiro: ABNT, 2011.

FORRESTER RESEARCH. **Low-Code Platforms For Business Developers**. Cambridge, MA: Forrester, 2014. Disponível em: <https://www.forrester.com>. Acesso em: 15 mar. 2026.

KRUCHTEN, Philippe. **The 4+1 View Model of Architecture**. IEEE Software, v. 12, n. 6, p. 42-50, nov. 1995.

MCCABE, Thomas J. **A Complexity Measure**. IEEE Transactions on Software Engineering, v. SE-2, n. 4, p. 308-320, dez. 1976.

RICHARDSON, Clay; RYMER, John R. **The Forrester Wave™: Low-Code Development Platforms, Q2 2014**. Cambridge, MA: Forrester Research, 2014.

WEILL, Peter; ROSS, Jeanne W. **Governança de TI**: como as empresas com melhor desempenho administram os direitos de decisão e a responsabilidade para obter resultados superiores. São Paulo: M. Books, 2006.